# LIBVIRT-BASED CLOUD AUTOSCALING

Programming Assignment 1 · Experimental Results and System Evaluation

**Project objective.** Implement a small cloud-management system using the libvirt API. A client sends CPU-intensive requests to a server VM; a host-side autoscaler monitors VM CPU utilization and starts a second VM when sustained overload is detected. The client then distributes requests across the available servers.

| | |
|---|---|
| **Environment** | Ubuntu 24.04 host; QEMU/KVM and libvirt; Python client/server/autoscaler |
| **VM configuration** | Two Ubuntu VMs, each configured with 1 vCPU and 1 GiB RAM |
| **Scale-out rule** | CPU threshold 80%; 5-second monitoring interval; 3 consecutive high samples |
| **Observed outcome** | Average latency reduced by 74.5%; throughput increased by 42.3% in the recorded run |

Prepared from the experiment measurements and autoscaler log available in the project context. The figures in this report were reconstructed from those recorded values; the original graph image files were not available in the current workspace.

# 1. System Design

The system demonstrates horizontal scaling at the virtual-machine level. It begins with one active server VM. The autoscaler collects CPU-utilization measurements through libvirt APIs, rather than invoking command-line monitoring tools. When the configured overload condition persists, it starts the second pre-defined VM, checks that its server endpoint is healthy, and notifies the client to add the new server to its request pool.

## Core components

**Server (server.py):** runs inside each VM and exposes a health endpoint and a CPU-intensive compute endpoint on port 8000.

**Client (client.py):** generates low- or high-rate workloads, measures request outcomes and latency, and distributes requests across known servers using round-robin selection. It exposes a local control endpoint so the autoscaler can register a new server.

**Autoscaler (autoscaler.py):** connects to libvirt, monitors VM CPU utilization, applies a threshold with consecutive-sample confirmation, starts VM2, checks readiness, and informs the client.

**Analysis and plots:** analyze_metrics.py summarizes client measurements; plot_metrics.py creates graphs from the collected metrics.

# 2. Experimental Method

The recorded experiment used a low-load phase followed by a high-load phase. The high-load generator was configured for 12 requests per second. The autoscaler checked CPU utilization at approximately five-second intervals and triggered scale-out after three consecutive samples at or above 80%. After VM2 became healthy and was registered with the client, requests could be served by either VM.

The autoscaler log records VM1 reaching 100% CPU in three consecutive samples before the scale-out event. Following VM2 startup, both VMs handled requests and their CPU utilization later fell as the workload subsided. There is a gap in elapsed time around the startup event, consistent with the time required to boot the VM and wait for service readiness.

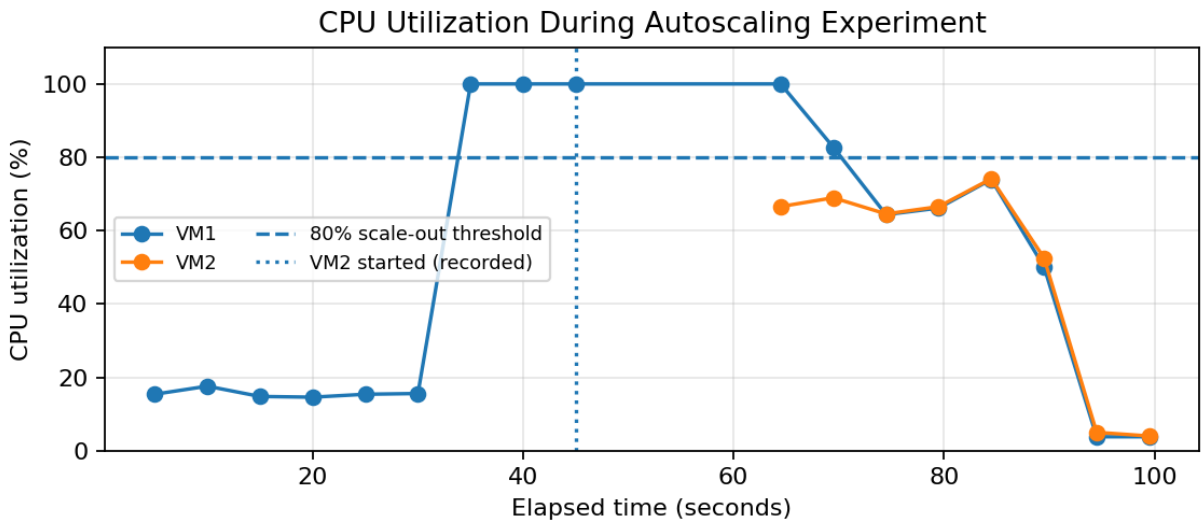



Figure 1. CPU utilization timeline reconstructed from the recorded autoscaler samples. The dotted vertical line marks the logged VM2-start event; the 80% line is the configured threshold.

## 3. Results

The table compares the recorded high-load measurements before and after scale-out. These are observed values from one run, not guaranteed performance figures for every run.

| Metric | Before scale-out | After scale-out |
|---|---|---|
| Requests recorded | 257 | 370 |
| Successful requests | 257 | 370 |
| Success rate | 100% | 100% |
| Average latency | 2,584.18 ms | 658.80 ms |
| Median latency | 1,727.90 ms | 113.85 ms |
| Observed throughput | 8.94 req/s | 12.72 req/s |
| Requests served by VM1 | 257 | 202 |
| Requests served by VM2 | 0 | 168 |

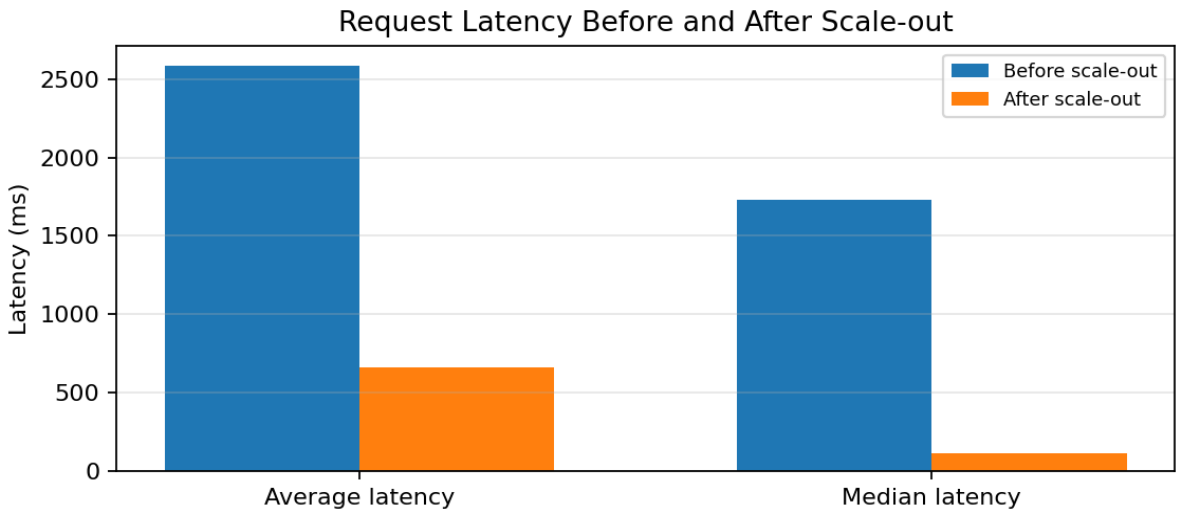



Figure 2. Average and median request latency both decreased after the second VM joined the client's server pool.

## 4. Throughput and Load Distribution

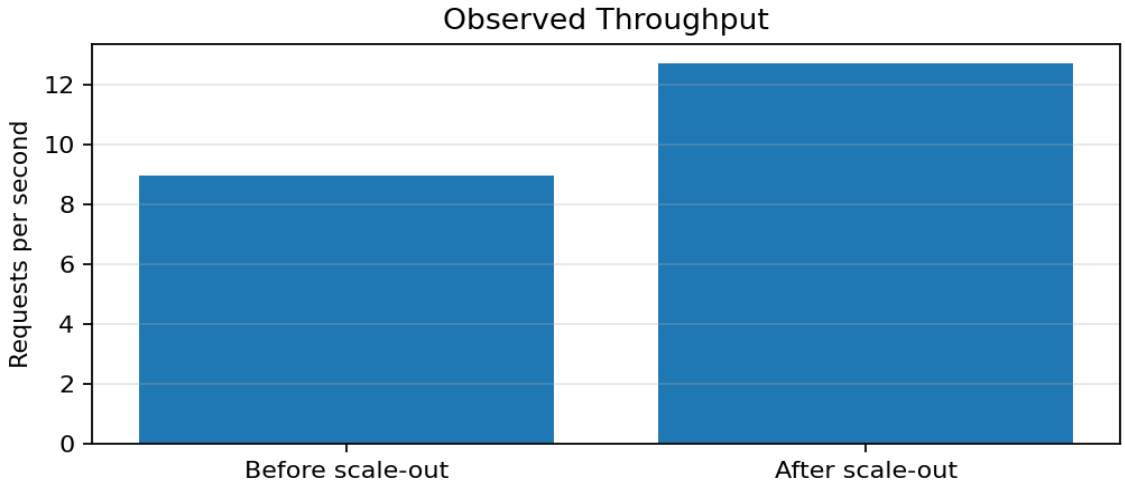



Figure 3. Observed throughput increased from 8.94 to 12.72 requests per second after scale-out.

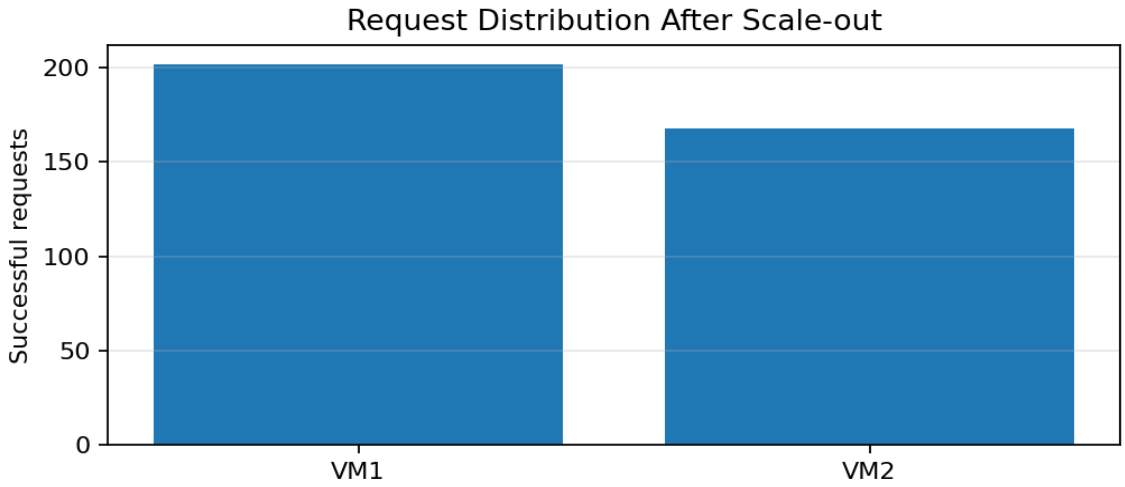



Figure 4. After scale-out, 202 recorded requests were served by VM1 and 168 by VM2, demonstrating that the client used the newly registered server.

## 5. Interpretation

**Latency:** average latency decreased by 74.5% and median latency decreased by 93.4%. The larger reduction in median latency suggests that many requests completed substantially faster after the workload was spread across two VMs.

**Throughput:** observed throughput increased by 42.3%. The second server provided additional capacity under the tested workload.

**Correctness and distribution:** the recorded requests in both comparison windows were successful, and the post-scale-out counts show requests served by both VMs. This supports that the scale-out notification and round-robin distribution path operated during the experiment.

# 6. Limitations and Reproducibility

The reported numbers represent a single recorded experiment. Performance depends on host contention, VM startup time, request complexity, client concurrency, network overhead, and the duration of each measurement window. The before/after windows contain different request counts and should be interpreted as an observed demonstration rather than a controlled statistical benchmark.

### For a repeatable demonstration:

1. Ensure VM1 is running and VM2 is defined but powered off before starting the autoscaler.
2. Start the autoscaler on the host, then start the client targeting VM1.
3. Run low load first, then switch to high load and observe the CPU log.
4. Confirm that the autoscaler starts VM2, waits for its health endpoint, and registers it with the client.
5. Check the client metrics and confirm that both server addresses appear in the request distribution.
6. Stop the client and autoscaler cleanly, then regenerate the analysis and plots from the CSV files.

# 7. Conclusion

The implementation demonstrates the main autoscaling workflow required for a libvirt-based cloud-management system: direct libvirt monitoring, threshold-based overload detection, VM startup, readiness checking, client notification, and round-robin request distribution. In the recorded high-load run, scaling from one server VM to two coincided with a substantial reduction in request latency and a measurable throughput increase. Further repeated runs with fixed-duration measurement windows would improve the strength of the performance comparison.

---

### Appendix: Recorded configuration

Host OS: Ubuntu 24.04 · VM1 libvirt domain: ubuntu24.04 (guest hostname server1; 192.168.122.236) · VM2 libvirt domain: server2 (192.168.122.237) · Each VM: 1 vCPU and 1 GiB RAM · Server port: 8000 · CPU threshold: 80% · Monitoring interval: 5 seconds · Confirmation: 3 consecutive high samples · High-load generator configuration: 12 requests/second.

Note: Replace or supplement the reconstructed charts with the original exported plots if exact graph styling or raw-data validation is required for submission.